• Experiment No.: 8

1. Title: Access and validate form fields using events; design a gym admission form with live input checks.

2. Software/Tools Required:

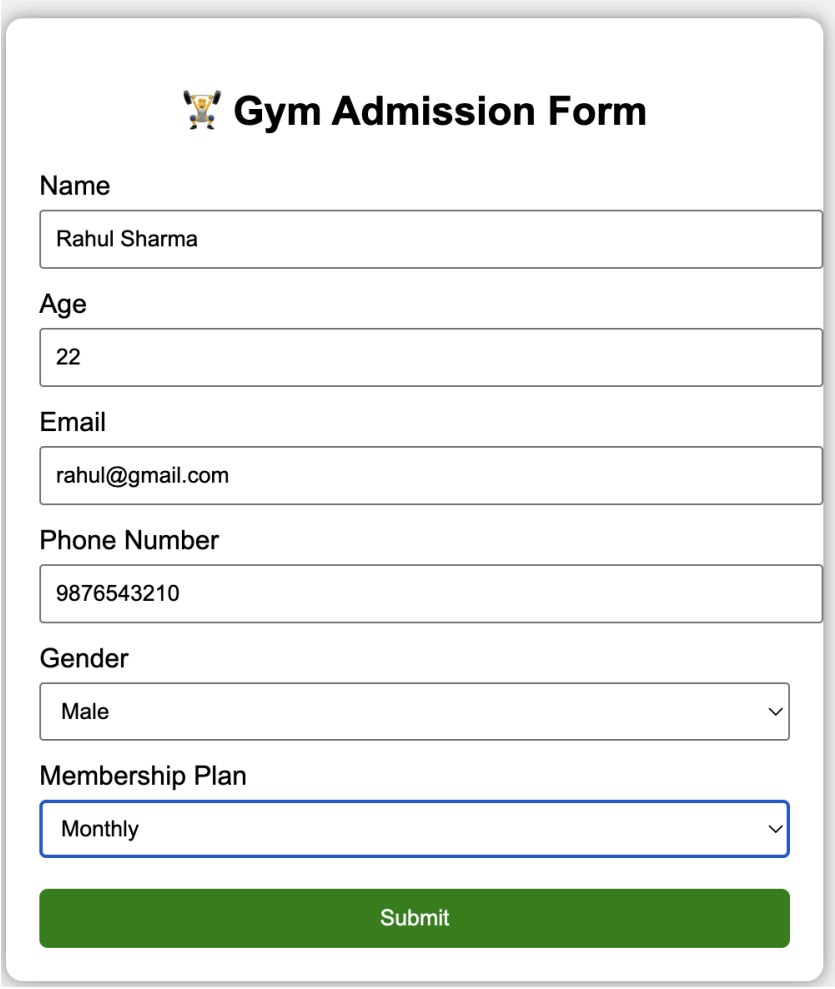
- Visual Studio Code (VS Code)
- Web Browser (Google Chrome/Microsoft Edge)
- JavaScript
- HTML

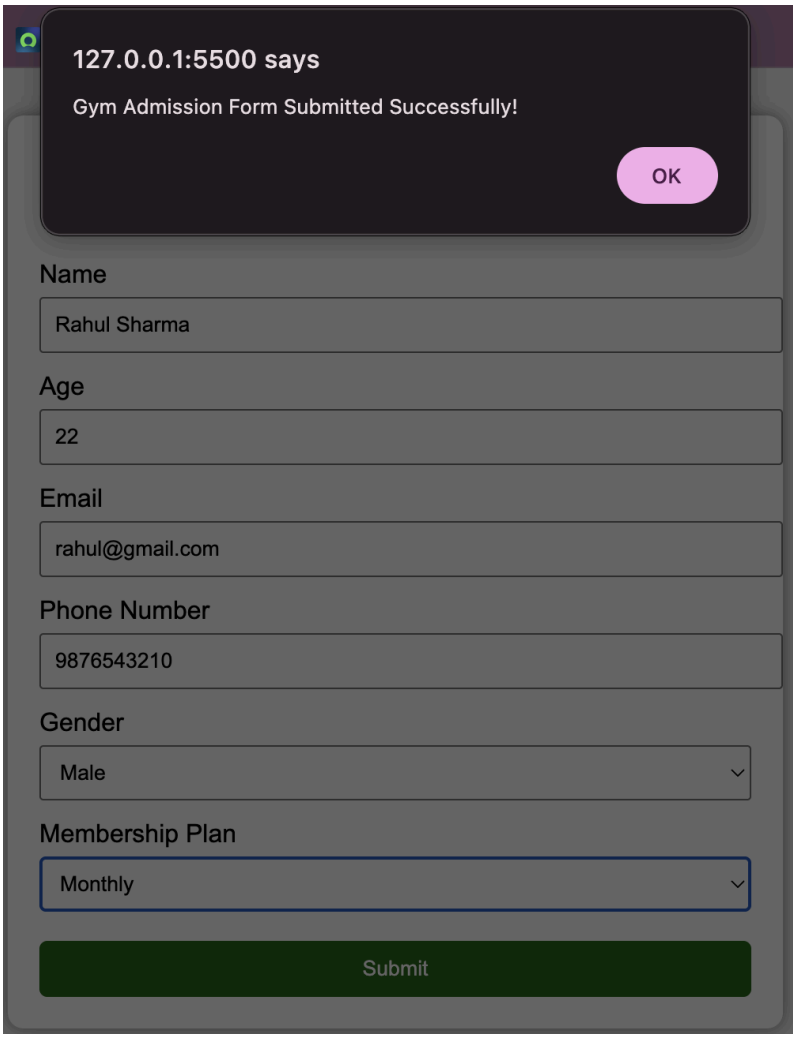
3. Theory: Form validation is the process of checking whether the user has entered valid and complete information before submitting a form. JavaScript provides events such as input, blur, change, and submit to validate form fields while the user is entering data.

In this experiment, a Gym Admission Form is created to collect details such as Name, Age, Email, Phone Number, Gender, and Membership Plan. Live validation is performed using JavaScript events. If the user enters invalid data, an error message is displayed immediately. This improves user experience by preventing incorrect data from being submitted.

The experiment demonstrates the use of DOM manipulation, event handling, regular expressions (Regex), and form validation techniques used in modern web applications.

4. Output:


Gym Admission Form
Name
Rahul Sharma
Age
22
Email
rahul@gmail.com
Phone Number
9876543210
Gender
Male
Membership Plan
Monthly
Submit



5. Result/Conclusion: This experiment demonstrated how to access and validate form fields using JavaScript events. A Gym Admission Form was developed with live input validation for the user's name, age, email, phone number, gender, and membership plan. The program provided immediate feedback for invalid inputs and prevented form submission until all fields were correctly filled. This experiment improved understanding of event handling, DOM manipulation, regular expressions (Regex), and client-side form validation, which are essential techniques for creating secure and user-friendly web applications.

• Experiment No.: 9

1. Title: Implement localStorage and sessionStorage to save user preferences like theme settings.

2. Software/Tools Required:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome/Microsoft Edge)
- JavaScript
- HTML

3. Theory: Web Storage is a feature of HTML5 that allows web applications to store data in the browser without using a database. It provides two storage mechanisms: localStorage and sessionStorage.

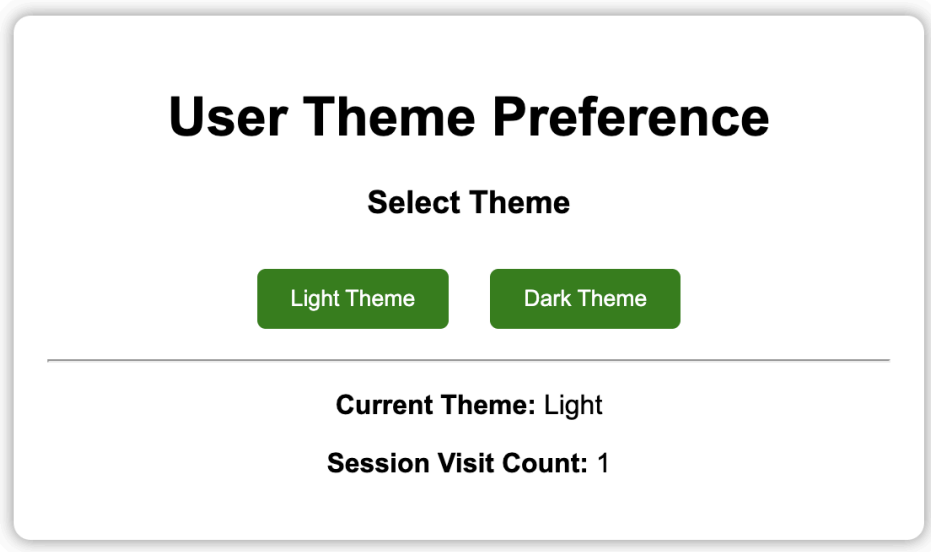
- localStorage stores data permanently in the browser until it is manually removed by the user or through JavaScript. The stored data remains available even after the browser is closed and reopened.

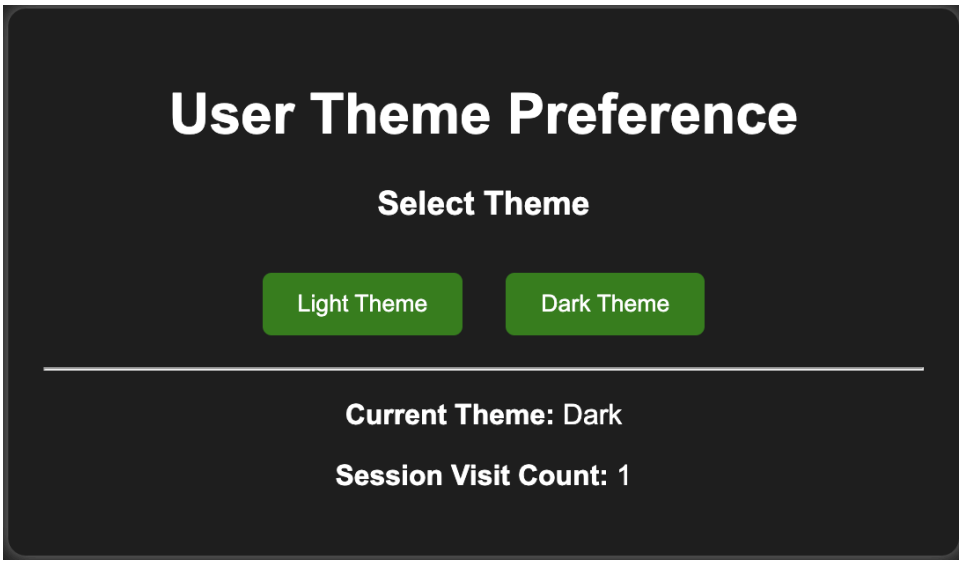
- sessionStorage stores data only for the current browser session. The data is automatically deleted when the browser tab or window is closed.

Both storage methods store data as key-value pairs and support methods such as setItem(), getItem(), removeItem(), and clear().

In this experiment, a webpage allows the user to select a Light or Dark theme. The selected theme is saved in localStorage, so it is remembered even after reopening the browser. The number of times the page is visited during the current session is stored in sessionStorage.

4. Output:


User Theme Preference
Select Theme
Light Theme
Dark Theme
Current Theme: Light
Session Visit Count: 1


User Theme Preference
Select Theme
Light Theme
Dark Theme
Current Theme: Dark
Session Visit Count: 1

5. Result/Conclusion: This experiment demonstrated the use of localStorage and sessionStorage in JavaScript to store user preferences and session data. The selected theme was saved in localStorage, allowing it to persist even after the browser was closed and reopened. The page visit count was stored in sessionStorage, making it available only during the current browser session. This experiment improved understanding of browser storage, data persistence, and user preference management, which are important concepts for developing interactive and personalized web applications.

- **Experiment No.: 10**

1. **Title:** Load and display JSON data using fetch() and jQuery $.getJSON(); render results dynamically in a table.

2. **Software/Tools Required:**
- Visual Studio Code (VS Code)
- Web Browser (Google Chrome/Microsoft Edge)
- JavaScript
- HTML

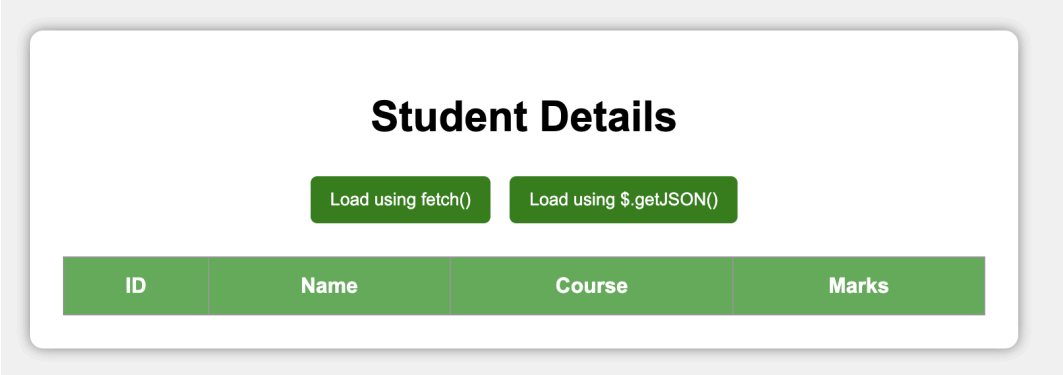
3. **Theory:** JSON (JavaScript Object Notation) is a lightweight data format used to store and exchange information between a server and a web application. JSON stores data as key-value pairs, making it easy to read and process.

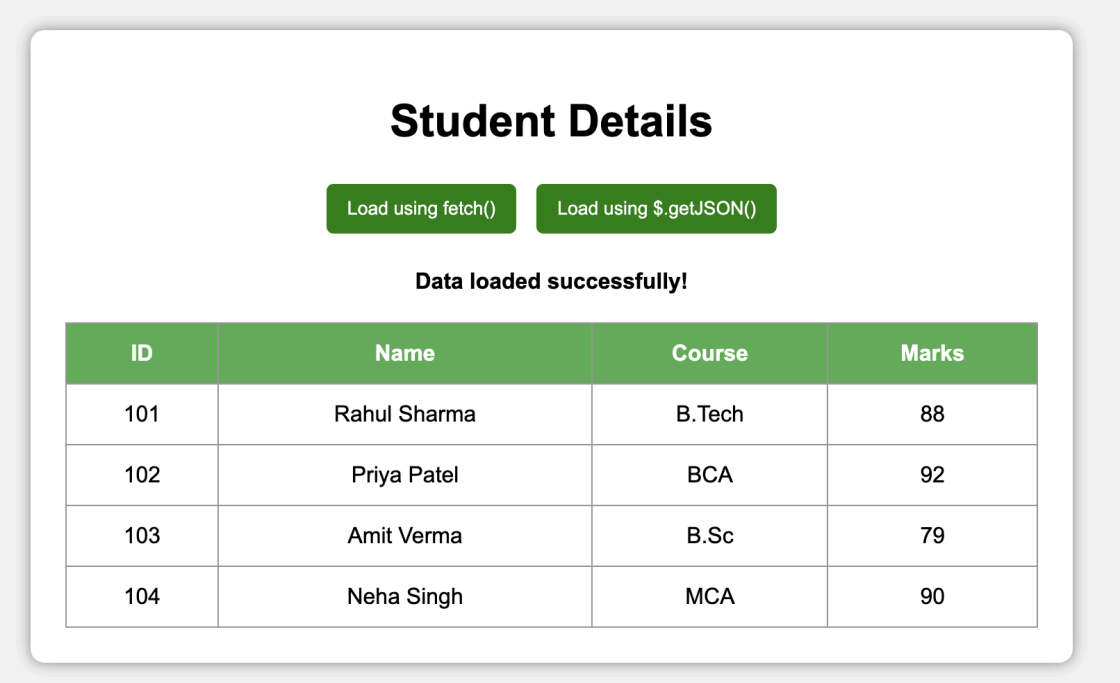
JavaScript provides the fetch() API to retrieve JSON data asynchronously from a local file or a web server. It returns a Promise, which is handled using .then() to process the response.

jQuery provides the $.getJSON() method, which is a simpler way to load JSON data. It automatically retrieves and converts JSON data into JavaScript objects.

In this experiment, student records are stored in a JSON file. The data is loaded using both fetch() and $.getJSON(), and the retrieved information is displayed dynamically in an HTML table using DOM manipulation.

4. Output:


Student Details
Load using fetch()
Load using $.getJSON()
ID
Name
Course
Marks


Student Details
Load using fetch()
Load using $.getJSON()
Data loaded successfully!
ID
Name
Course
Marks
101
Rahul Sharma
B.Tech
88
102
Priya Patel
BCA
92
103
Amit Verma
B.Sc
79
104
Neha Singh
MCA
90

5. Result/Conclusion: This experiment demonstrated how to load JSON data using the fetch() API and jQuery $.getJSON(). The retrieved JSON data was processed and displayed dynamically in an HTML table using DOM manipulation. The experiment improved understanding of JSON, AJAX requests, asynchronous programming, and dynamic content rendering, which are essential concepts for developing modern, data-driven web applications.